ShopSphere Scalability Audit

CSA06 CO1 Assessment Tool 1

Student: Hitesh **Register number:** 192524005 **Course:** CSA06 – Design and Analysis of Algorithms **Question:** IT application-based problem solving

Objective

This report examines the five ShopSphere back-end modules as the catalogue grows from 100,000 to 1,000,000 products. It identifies each dominant operation, derives the requested counts and recurrences, and compares estimated processing times with the 10-minute nightly window. The main conclusion is that the duplicate detector and the current recommendation design need changes before the catalogue reaches one million products.

Assumptions and method

- One basic operation per second-rate unit is assumed to run at 100,000,000 operations per second, as given in the question. Estimated seconds equal operation units divided by 100,000,000.
- The exact M1 and M2 counts use the input n = 1,000,000.
- For recurrence estimates, n is rounded up to the next power of two, 1,048,576 = 2^{20}. This makes the recursive levels integral and gives a slightly conservative estimate for one million items.
- For the recurrence estimates, the base count is T(1) = 1. For M4 and M5, the count includes the work at internal nodes and the one-unit base cases.
- These are mathematical estimates under the stated rate, not a benchmark of processing one million real products. The C++ program calculates the estimates; its own runtime is not the platform runtime.

(a) Understand the problem

The input size n is the number of products or records being processed. A basic operation is the repeated unit whose count gives the dominant growth rate.

Module	Input and dominant operation	Where it occurs and growth
M1 Duplicate-Listing Detector	n products; compare a pair's title and seller	The equality check occurs for every unordered pair. There are n(n − 1)/2 comparisons, so the count grows as $\Theta(n^2)$.
M2 API Rate Limiter	n requests; execute `Audit(Request[j])`	It runs for windows of sizes 1, 2, 4, … below n. The geometric sum is less than 2n, so it grows as Θ(n), not Θ(n log n).
M3 S1 Product Search	n unsorted products; compare the search key with a product	A sequential scan may inspect every product, so the worst-case count grows as Θ(n).
M3 S2 Product Search	n sorted products; compare with the middle key	Each comparison discards half the remaining index. The count grows as Θ(log n).
M4 Recommendation Engine	n customer-product records; combine partial results	The current design has four recursive parts of size n/2 and linear combine work, giving $\Theta(n^2)$. Removing replication gives two parts and Θ(n log n).
M5 Sales Analytics Engine	n sales records; sort at every recursive level	The recurrence adds n log n work per level, giving $\Theta(n \log^2 n)$. The separate report-generator function is analyzed in part (f).

(b) Iterative modules

M1 exact comparisons

For each outer-loop index i, the inner loop compares product i with the n − i − 1 later products:

```
C_M1(n) = Σ(i=0 to n-2) (n-i-1)
        = (n-1) + (n-2) + … + 2 + 1
        = n(n-1)/2
```

At n = 1,000,000:

```
C_M1 = 1,000,000 × 999,999 / 2
     = 499,999,500,000 comparisons
```

The time complexity is $\Theta(n^2)$. The loops use Θ(1) auxiliary space, excluding the product catalogue itself.

M2 exact Audit count

The outer-loop values are powers of two smaller than n. Let k = ⌈log₂ n⌉. The audited window sizes are 1, 2, 4, …, 2^(k−1):

```
C_M2(n) = 1 + 2 + 4 + … + 2^(k-1)
        = 2^k - 1
        = 2^ log₂ n  - 1
```

For n = 1,024, k = 10, so the sizes are 1 through 512 and the count is $2^{10} - 1$ = **1,023**. The program prints this verification. The claim Θ(n log n) is incorrect: the sum is less than 2n and at least n − 1, so the tight bound is **Θ(n)**. The routine uses Θ(1) auxiliary space.

For n = 1,000,000, k = 20 and the count is $2^{20} - 1$ = **1,048,575**.

(c) Solve S2 by backward substitution

The recurrence is T(n) = T(n/2) + 1, with T(1) = 1. Assume n = 2^k for the exact derivation:

```
T(n)       = T(n/2) + 1
           = T(n/4) + 2
           = T(n/8) + 3
           = …
           = T(n/2^k) + k
```

Stop when n/2^k = 1. Then k = $\log_2 n$, so:

```
T(n) = T(1) + log₂ n = 1 + log₂ n
```

Thus S2 takes Θ(log n) time. An iterative binary search uses Θ(1) auxiliary space. A recursive binary search uses Θ(log n) call-stack space. For an arbitrary integer-sized index, the worst number of key comparisons is ⌊log₂ n⌋ + 1; at n = 1,000,000, that is 20 comparisons. The recurrence's exact closed form above assumes a power-of-two input.

(d) Master Theorem and recursion tree

M4 current design

For T(n) = 4T(n/2) + n:

Quantity	Value
a	4
b	2
f(n)	n
n^(log_b a)	n^(log₂4) = n^2
Comparison	f(n) = O(n^(2−1)); it is polynomially smaller
Case and result	Master Theorem Case 1; $\Theta(n^2)$

For n = 2^k, level i has 4^i subproblems of size n/2^i. Each subproblem contributes n/2^i combine operations, so the whole level costs 4^i(n/2^i) = n·2^i. The internal levels cost:

```
Σ(i=0 to k−1) n·2^i = n(2^k − 1) = n(n − 1)
```

There are 4^k = n^2 leaves, each with one base operation. Thus T(n) = n(n − 1) + n^2 = **$2n^2 - n$**. At n = 1,048,576 this gives **2,199,022,206,976** operation units.

Doubling n multiplies this dominant cost by about 4. The estimate at the padded size 1,048,576 is $2n^2 - n$ = 2,199,022,206,976 operation units when T(1) = 1.

M4 proposed design

For T(n) = 2T(n/2) + n:

Quantity	Value
a	2
b	2
f(n)	n
n^(log_b a)	n^($\log_2$2) = n
Comparison	f(n) = Θ(n^(log_b a))
Case and result	Master Theorem Case 2; Θ(n log n)

At level i there are 2^i subproblems of size n/2^i, so the total combine work at that level is n. There are k = $\log_2 n$ internal levels, for nk combine operations, and n leaves with one base operation each. Therefore T(n) = nk + n = **$n(\log_2 n + 1)$**. At n = 1,048,576 this gives **22,020,096** operation units.

When n doubles, the dominant work grows by roughly a factor of 2, with a small logarithmic increase: T(2n)/T(n) ≈ 2($\log_2 n$ + 1)/$\log_2 n$. This is a substantial improvement over the current quadratic design.

M5 sales analytics

For T(n) = 2T(n/2) + n $\log_2 n$, a = 2, b = 2 and n^(log_b a) = n. The added work is n log n, which has the same polynomial exponent and one extra logarithmic factor. Under the extended Master Theorem (or by a recursion tree),

```
T(n) = Θ(n log² n)
```

At recursion-tree level i, the total local work is n $\log_2$(n/2^i) = n(k − i). Summing these values over the k = $\log_2 n$ internal levels gives:

```
Σ(i=0 to k−1) n(k − i) = n(1 + 2 + … + k)
                       = nk(k + 1)/2
```

The n leaves contribute another n base operations. Thus T(n) = n(k(k + 1)/2 + 1), which is Θ(n $\log^2 n$). At n = 1,048,576 (k = 20), the estimate is **221,249,536** operation units.

(e) Best, average and worst cases

S1 sequential search

- **Best:** the product is first in the catalogue: 1 comparison.
- **Average when present:** if each position is equally likely, (1 + 2 + … + n)/n = (n + 1)/2 comparisons.
- **Worst:** the product is last or absent: n comparisons.

Let p be the probability that the product exists. If it exists, use the average successful count (n + 1)/2; if it does not, the scan checks all n products:

```
E[C_S1] = p(n + 1)/2 + (1 − p)n
        = n(1 − p/2) + p/2
```

For n = 1,000,000 and p = 0.5:

```
E[C_S1] = 0.5 × 500,000.5 + 0.5 × 1,000,000
        = 750,000.25 comparisons
```

S2 binary search

- **Best:** the target equals the first middle key: 1 comparison.
- **Average:** Θ(log n) comparisons for a balanced sorted index; the exact mean depends on the midpoint convention and catalogue size.
- **Worst:** the target is at the deepest search position or absent: $\lfloor \log_2 n \rfloor + 1$ comparisons. At n = 1,000,000, this is 20.

(f) Prove the report-generator bound

The function is $f(n) = 3n^2 + 10n \log_2 n + 50$. We use the definition of $\Theta(n^2)$: find positive constants c_1, c_2 and n_0 such that $c_1 n^2 \le f(n) \le c_2 n^2$ for every $n \ge n_0$.

For $n \ge 2$, $\log_2 n \le n$, and $1 \le n^2$. Therefore:

```
3n² ≤ f(n)
f(n) = 3n² + 10n log₂n + 50
     ≤ 3n² + 10n² + 50n²
     = 63n²
```

Choose **$c_1 = 3$, $c_2 = 63$ and $n_0 = 2$**. Both inequalities hold for all $n \ge 2$, so $f(n) = \Theta(n^2)$.

To show f(n) is not O(n), divide by n:

```
f(n)/n = 3n + 10 log₂n + 50/n ≥ 3n
```

The right-hand side grows without bound. No fixed constant c can satisfy $f(n) \le cn$ for all sufficiently large n. Hence f(n) is **not O(n)**.

(g) One-million-item estimates and decision

The times below use 100,000,000 operations/second. For recursive rows, n is padded to 1,048,576 and T(1) = 1. The operation units in the M4 and M5 rows follow the recurrences, including leaf base cases.

Module/design	Estimated operation units	Estimated time	Within 10 minutes?
M1 duplicate detector	499,999,500,000	1 h 23 min 19.995 s	No
M2 rate limiter	1,048,575	0.0105 s	Yes
S2 binary search, worst case	20	0.0000002 s	Yes
M4 current design	2,199,022,206,976	6 h 6 min 30.222 s	No
M4 proposed design	22,020,096	0.2202 s	Yes
M5 sales analytics	221,249,536	2.2125 s	Yes

M1 and the current M4 design exceed the 10-minute window and should be redesigned. For M1, index products by a composite key such as `(seller, normalized title)` so duplicate checks use a hash table or database index; expected processing becomes Θ(n), with Θ(n) additional storage. For M4, remove replicated recursive work and use the proposed two-way split with linear merging, reducing the recurrence to Θ(n log n). M2 and S2 fit easily. M5 also fits under the stated model; it can be monitored as data grows, and avoiding repeated sorting at every recursive level could reduce its cost to Θ(n log n).

The separate function f(n) from part (f) is quadratic, but the question does not provide a constant factor linking that profile to the M5 recurrence. It is therefore proved asymptotically in part (f), not added as a separate row to the M5 estimate.

Program and verification

The C++17 program in `../Source_Code/scalability_audit.cpp` calculates the M1 count, M2 geometric sum, binary-search comparison bound, and the padded recursive estimates. It also checks the M2 count at n = 1,024. It was compiled with `g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic` using LLVM-MinGW (Clang 23.1.1), then executed successfully. The compiler details are in `compile_log.txt`; the captured output is in `program_output.txt`.

Source code screenshots

These images are rendered directly from the submitted source file, with line numbers, for readability.

Source code, part 1

C++ source code — part 1

scalability_audit.cpp | lines 1–30 | rendered from submitted source

```cpp
#include <cmath>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>

using Count = std::uint64_t;
constexpr long double OPERATIONS_PER_SECOND = 100'000'000.0L;

Count duplicateComparisons(Count n) {
    return n * (n - 1) / 2;
}

Count auditOperations(Count n) {
    Count total = 0;
    for (Count window = 1; window < n; window *= 2) {
        total += window;
        if (window > UINT64_MAX / 2) break;
    }
    return total;
}

Count nextPowerOfTwo(Count n) {
    Count power = 1;
    while (power < n) power *= 2;
    return power;
}

unsigned log2OfPowerOfTwo(Count n) {
```

Source code, part 2

C++ source code — part 2

scalability_audit.cpp | lines 31–60 | rendered from submitted source

```
    unsigned levels = 0;
    while (n > 1) {
        n /= 2;
        ++levels;
    }
    return levels;
}

Count binarySearchWorstComparisons(Count n) {
    Count comparisons = 1;
    while (n > 1) {
        n /= 2;
        ++comparisons;
    }
    return comparisons;
}

std::string withCommas(Count value) {
    std::string digits = std::to_string(value);
    for (int pos = static_cast<int>(digits.size()) - 3; pos > 0; pos -= 3) {
        digits.insert(static_cast<std::size_t>(pos), ",");
    }
    return digits;
}

std::string humanDuration(long double seconds) {
    if (seconds >= 3600.0L) {
        const auto hours = static_cast<Count>(seconds / 3600.0L);
        seconds -= hours * 3600.0L;
        const auto minutes = static_cast<Count>(seconds / 60.0L);
```

Source code, part 3

C++ source code — part 3

scalability_audit.cpp | lines 61–90 | rendered from submitted source

```
61          seconds -= minutes * 60.0L;
62          std::ostringstream out;
63          out << hours << " h " << minutes << " min " << std::fixed
64              << std::setprecision(3) << seconds << " s";
65          return out.str();
66      }
67      if (seconds >= 60.0L) {
68          const auto minutes = static_cast<Count>(seconds / 60.0L);
69          seconds -= minutes * 60.0L;
70          std::ostringstream out;
71          out << minutes << " min " << std::fixed << std::setprecision(3)
72              << seconds << " s";
73          return out.str();
74      }
75      std::ostringstream out;
76      out << std::scientific << std::setprecision(3) << seconds << " s";
77      return out.str();
78  }
79
80  void printRow(const std::string& module, Count operations) {
81      const long double seconds = operations / OPERATIONS_PER_SECOND;
82      std::cout << std::left << std::setw(25) << module << std::right
83                << std::setw(20) << withCommas(operations) << "  "
84                << humanDuration(seconds) << '\n';
85  }
86
87  int main() {
88      constexpr Count n = 1'000'000;
89      constexpr Count auditCheckN = 1'024;
90      const Count paddedN = nextPowerOfTwo(n);
```

Source code, part 4

C++ source code — part 4

scalability_audit.cpp | lines 91–120 | rendered from submitted source

```
91      const unsigned levels = log2OfPowerOfTwo(paddedN);
92
93      const Count m1 = duplicateComparisons(n);
94      const Count m2 = auditOperations(n);
95      const Count s2Worst = binarySearchWorstComparisons(n);
96
97      // T(1)=1; the recursive estimates use a power-of-two input size.
98      const Count m4Current = 2 * paddedN * paddedN - paddedN;
99      const Count m4Proposed = paddedN * (levels + 1);
100     const Count m5 = paddedN * (levels * (levels + 1) / 2 + 1);
101
102     std::cout << "CSA06 CO1 AT1 - ShopSphere Scalability Audit\n";
103     std::cout << "Student: Hitesh | Register No.: 192524005\n";
104     std::cout << "Rate: 100,000,000 basic operations per second\n";
105     std::cout << "Recursive estimates: next power of two = " << withCommas(paddedN)
106               << " (" << levels << " levels); base case T(1)=1\n\n";
107
108     std::cout << "M2 verification at n = " << withCommas(auditCheckN)
109               << ": " << withCommas(auditOperations(auditCheckN))
110               << " Audit operations (expected 1,023)\n";
111     std::cout << "M2 at n = " << withCommas(n) << ": " << withCommas(m2)
112               << " Audit operations\n\n";
113
114     std::cout << std::left << std::setw(25) << "Module / design" << std::right
115               << std::setw(20) << "Operation units" << "  Estimated time\n";
116     std::cout << std::string(72, '-') << '\n';
117     printRow("M1 duplicate detector", m1);
118     printRow("M2 rate limiter", m2);
119     printRow("S2 binary search (worst)", s2Worst);
120     printRow("M4 current design", m4Current);
```

Source code, part 5

C++ source code — part 5

scalability_audit.cpp | lines 121–130 | rendered from submitted source

```
121       printRow("M4 proposed design", m4Proposed);
122       printRow("M5 sales analytics", m5);
123
124       const long double s1Average = 0.5L * ((n + 1) / 2.0L) + 0.5L * n;
125       std::cout << "\nS1 average at n=1,000,000 and p=0.5: " << std::fixed
126                 << std::setprecision(2) << s1Average << " comparisons\n";
127       std::cout << "10-minute decision: M1 and current M4 exceed the window.\n";
128       std::cout << "Program completed successfully.\n";
129       return 0;
130   }
```

Program output screenshot

This image is a readable rendering of the output captured from the executed program.

Captured program output

Program output

Captured stdout from the compiled C++ program | program_output.txt

```
CSA06 CO1 AT1 - ShopSphere Scalability Audit
Student: Hitesh | Register No.: 192524005
Rate: 100,000,000 basic operations per second
Recursive estimates: next power of two = 1,048,576 (20 levels); base case T(1)=1

M2 verification at n = 1,024: 1,023 Audit operations (expected 1,023)
M2 at n = 1,000,000: 1,048,575 Audit operations

Module / design              Operation units  Estimated time
-------------------------------------------------------------------------
M1 duplicate detector         499,999,500,000  1 h 23 min 19.995 s
M2 rate limiter                     1,048,575  1.049e-02 s
S2 binary search (worst)                   20  2.000e-07 s
M4 current design           2,199,022,206,976  6 h 6 min 30.222 s
M4 proposed design                 22,020,096  2.202e-01 s
M5 sales analytics                221,249,536  2.212e+00 s

S1 average at n=1,000,000 and p=0.5: 750000.25 comparisons
10-minute decision: M1 and current M4 exceed the window.
Program completed successfully.
```

Analysis and documentation screenshots

The report itself contains the full derivations and run instructions. These snapshots highlight the analysis and documentation sections.

Analysis summary

Analysis summary

Rendered excerpt from Report.md | full derivations are in the report

```
M1 pair comparisons: n(n−1)/2 = 499,999,500,000 at n = 1,000,000.
M2 windows: 1 + 2 + 4 + … + 2^(k−1) = 2^k − 1, k = ceil(log₂ n).
At n = 1,024, M2 performs 1,023 Audit operations; therefore its bound is Θ(n).
S2 substitution: T(n) = T(n/2) + 1 = T(1) + log₂ n = 1 + log₂ n.
M4 current: 4T(n/2)+n → Θ(n²); proposed: 2T(n/2)+n → Θ(n log n).
M5 recursion tree: n(log n + … + 1) + Θ(n) = Θ(n log² n).
For f(n), choose c₁=3, c₂=63, n₀=2; thus 3n² ≤ f(n) ≤ 63n².
At one million items, M1 and current M4 exceed the 10-minute window.
Recommendation: index M1 by seller/title and remove M4's replicated recursion.
```

Documentation and run instructions

Documentation and run instructions

Rendered excerpt from the CO1_AT1 README and Report

```
Files included in this question folder:
  Source_Code/scalability_audit.cpp
  Report/Report.md
  Report/program_output.txt
  Report/Evidence/*.png

Compile from the Source_Code folder:
  g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic scalability_audit.cpp -o scalability_audit

Run on Linux/macOS:
  ./scalability_audit

Run on Windows after compilation:
  .\scalability_audit.exe

The program assumes 100,000,000 operations per second, checks M2 at n=1,024,
and estimates the requested modules at n=1,000,000.
```

Code of conduct

The question paper requires the student to certify the submission and sign it. Review the course policy and complete this section personally. Do not sign the declaration unless it is accurate for your work and the assistance used.

- Student name: Hitesh
- Register number: 192524005
- Student signature: _____
- Date: _____